we assume that any operation ∈ { addit-, mult, ... } take $O(1)$ time

Divide and Conquer

Generic strategy

1. Divide the input into some # partitions (usually 2)
2. — Recursive solve the smaller subproblems if size $\geq n_0$ else solve directly
3. Conquer If necessary "patch up" the soln to the subproblems

How do

running time / space?

$T(n)$: time for solving a problem of size n

$$T(n) = \begin{cases} \sum_{i=1}^{k} T(n_i) + \text{Cost of partition } \pi(n) \\ \quad + M(n) \quad \text{if } n > n_0 \\ \quad \text{merge step} \\ \text{where } \sum n_i \le n \\ T(n_0) = \end{cases}$$

Merge sort

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + O(1) + O(n)$$

say $n_0 = 2$ $\quad T(2) = 1$

$\Rightarrow$ $T(n)$

$T(n) = O(n \log n)$

$T(n) = \sqrt{n}\,T(\sqrt{n}) + O(n^{4/3} \log^2 n)$

$T(n) = ?$

"Master theorem"

$F(n) = F(n-1) + F(n-2)$

$F_0 = 0 \qquad F_1 = 1$

$F(n) \approx \phi^n$

ϕ is a constant related golden ratio which is an irrational

$|F(n)| \sim n \log \phi = n^b$

size

Show that the # recursive calls in the above computation $\sim F(n)$

$|n| = \log_n \text{bits} = b$

$$\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} F_{n-1} \\ F_{n-2} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} F_{n-2} \\ F_{n-3} \end{bmatrix}$$

$$\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1} \begin{bmatrix} F_1 \\ F_0 \end{bmatrix}$$

Multiplying 2 n bit nos

$$A = a_{n-1}\ a_{n-2}\ \cdots\ a_0$$

$$B = b_{n-1}\ b_{n-2}\ \cdots\ b_0$$

A × B will take $O(n^2)$ if we do the long multiplication

For faster methods

$$A = [a_{n-1}\ a_{n-2}\ \cdots\ a_{\frac{n}{2}}]\ [a_{\frac{n}{2}-1}\ \cdots\ a_0]$$

$$B = [b_{n-1}\ b_{n-2}\ \cdots\ b_{\frac{n}{2}}]\ [b_{n/2-1}\ \cdots\ b_0]$$

B_1 $n/2$ bits B_0 $n/2$ bits

$$A = A_1 \cdot 2^{n/2} + A_0$$

$$B = B_1 \cdot 2^{n/2} + B_0$$

$$A \times B = A_1 \times B_1 \cdot 2^n + \left(A_1 \times B_0 + B_1 \times A_0 \right) 2^{n/2} + A_0 \times B_0$$

An n bit multiplication can be expressed in terms of 4 $\frac{n}{2}$ bit multipl. + multiplying by 2^n and $2^{n/2}$ + 3 additions

$$M(n) = 4M\left(\frac{n}{2}\right) + O(n) + O(n)$$

$$= 4M\left(\frac{n}{2}\right) + O(n)$$

$$= cn$$

$$M(2) = O(1)$$

$$= 4\left(4 \cdot M\left(\frac{n}{4}\right) + c \cdot \frac{n}{2}\right) + cn$$

$$= 4^2 M\left(\frac{n}{4}\right) + 4c\frac{n}{2} + cn$$

$$= 2cn$$

$$= 4^i M\left(\frac{n}{2^i}\right) + \ldots$$

for $i = \log_2 n$, $4^{\log_2 n} \cdot c$ + other terms

$4^{\log_2 n} \cdot c$ ← itself is $n^2 \cdot c$

$M(n) \geq n^2$

← growing

Observation : we need to do fewer than 4 multiplication to get better than $O(n^2)$ to w $O(n^2)$ small

$$(A_1 + A_0) \otimes (B_1 + B_0)$$

$$= A_1 \times B_1 + A_1 \times B_0 + A_0 \times B_1 + A_0 \times B_0$$

$$(A_1 + A_0)(B_1 + B_0) \overset{\checkmark}{-} A_1 \otimes B_0 \overset{\checkmark}{-} B_1 \otimes A_0$$

$$= A_1 B_1 + A_0 B_0$$

Ignore this instead

$$(A_1 + A_0) \times (B_1 + B_0) - A_1 \times B_1 - A_0 \times B_0$$

$$= A_1 B_0 + B_1 A_0$$

— to be continued —

The modified recurrence

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n) = cn$$

Unfolding the recurrence the most significant term

$$\leq 3^{\log_2 n} = 3^{\frac{\log n}{\log 2} \times \frac{\log_2 3}{\log_2 3}}$$

$$= (3^{\log_2 3})^{\log_2 n}$$

$= \left(3^{\log_2 3}\right) \cdot \log_2 3$

$$T(n) \leq \alpha \cdot n^{\log_2 3} = n^{\log_2 3}$$

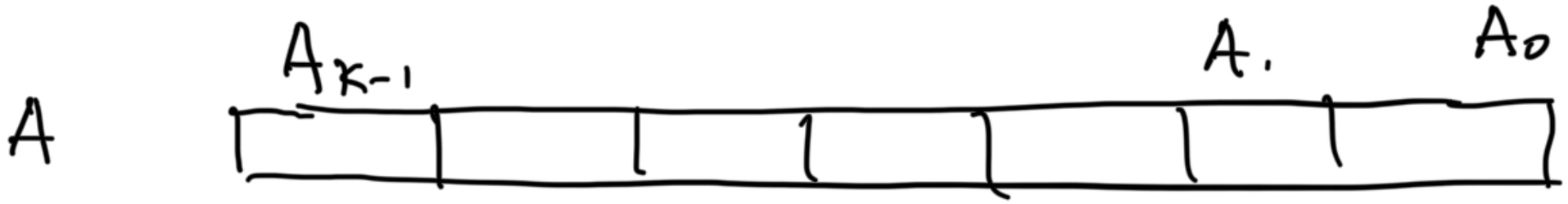
$$\sim \alpha \cdot n^{1.7}$$

We can verify by induction that the soln to the recurrence is $T(n) = \alpha n^{\log_2 3}$

Plug $T\left(\frac{n}{2}\right) : \alpha \left(\frac{n}{2}\right)^{\log_2 3}$

and show that $\alpha \left(\frac{n}{2}\right)^{\log_2 3} + cn.$

$$\leq \alpha\, n^{\log_2 3}$$



$$A \times B = \left[A_{k-1} 2^{(k-1)\cdot\frac{n}{k}} + A_{k-2} 2^{(k-2)\frac{n}{k}} \cdots A_0 \right] \times \left[B_{k-1} 2^{(k-1)\frac{n}{k}} + \cdots B_0 \right]$$

$$= A_{k-1} \times B_{k-1} 2^{2(k-1)\frac{n}{k}} + \cdots + A_0 B_0$$

By using $2^{n/k} = x$,

$$\left(A_{k-1} \cdot x^{k-1} + A_{k-2} x^{k-2} + \cdots A_0 \right) \times \left(B_{k-1} x^{k-1} + B_{k-2} x^{k-2} + \cdots B_0 \right)$$

evaluate the polynomials in $2k-1$ distinct points

$$= A_{R-1} \cdot B_{R-1} \cdot x^{2(R-1)} + (A_i B_{(R-1)}) x^{2(R-1)-1} + (\quad)$$

$$+ A_0 B_0 \cdot x^0$$

Recall if we know the value of a degree d polynomial in $d+1$ points, then we can compute the unique coefficients

Suppose f is a degree 3 poly.

and we know the values at $x_1, x_2 x_3, x_4$

$y_1, y_2 y_3 y_4$

$$a_3 x^3 + a_2 x^2 + a_1 x + a_0$$

$$a_3 x_1^3 + a_2 x_1^2 + a_1 x_1 + a_0 = y_1$$

$$a_3 x_2^3 + a_2 x_2^2 + a_1 x_2 + a_0 = y_2$$

$$\vdots$$

polynomial interpolation

$$\begin{bmatrix} x_1^3 & x_1^2 & x_1 & 1 \\ x_2^3 & x_2^2 & x_2 & 1 \\ & & & \end{bmatrix} \begin{bmatrix} a_3 \\ a_2 \\ a_1 \\ a_0 \end{bmatrix} = \begin{bmatrix} y_3 \\ y_2 \\ y_1 \\ y_0 \end{bmatrix}$$

We can choose $1, 2, 3, \cdots 2k-1$

Evaluate $A(1)\ A(2)\ A(3)\ \cdots\ A(2k-1)$

Evaluate $B(1)\ B(2)\ \cdots\ B(2k-1)$

$AB(1) = A(1)\ B(1)$ $\qquad AB(i) = \underbrace{A(i)}_{\frac{n}{k} \text{ bits}} \cdot B(i)$

$$T(n) = 2k-1\ T\left(\frac{n}{k}\right) + \overbrace{O(n)}^{\text{dependent on } k}$$

$O(n)$ must capture/subsume all the work for inverting a $K \times K$ matrix

when K is a fixed constant (not dependent on n)

By choosing a large K we can achieve $O(n^{1+\varepsilon})$

To go further, we choose $K \sim \sqrt{n}$ and use polynomial evaluations and interpolation on "roots of unity"